

TSOpt guide 2: Your first training set

Contents

1	Step 1: should I optimise at all?	1
2	Step 2: how many lines do I need?	3
3	Step 3: design the training set	4
4	Targeted and untargeted designs	6
5	Why robust CDmean is the default	7
6	Non-additive models: total or breeding value?	8
7	Other criteria	8
8	What exactly did it do? tso_explain()	9
9	Hand it over: tso_export() and tso_report()	10
10	The whole workflow in 8 lines	10
11	Next	10

Genotyping is cheap and phenotyping is not. TSOpt answers one question: **which of my genotyped candidates should get a field plot?** This guide goes through the core workflow on the example panel: is optimisation worth it here, how many lines do I need, which lines, why those, and how do I hand the list over.

We start from the object built in **TSOpt guide 1: Your data in**: 300 inbred lines, 1,000 SNPs, read, cleaned and turned into a relationship matrix in one line.

```
library(TSOpt)
ex <- function(f) system.file("extdata", f, package = "TSOpt")
tp <- tso_data(ex("example_panel.vcf.gz"))
ids <- tp$units$id
```

Suppose we can afford 60 plots, a fifth of the panel.

1 Step 1: should I optimise at all?

An optimised training set is not always much better than a random one. On a panel with little structure and many markers, random sampling is close to the best you can do, and it is better to know that before investing effort. `tso_diagnose()` asks the question directly.

```
dg <- tso_diagnose(tp, n = 60)
dg
#> TSOpt diagnosis · 300 candidates · planned n = 60 · h2 = 0.5
#> Verdict          AMBER expected accuracy gain over random at n = 60: 4.9%
```

```
#> Advice      Optimisation helps a little; tso_design() is still the better choice,
↳ but random sampling would not be a disaster. The panel has discrete groups:
↳ tso_design() will stratify automatically.
#> Structure    discrete groups (4 groups, silhouette 0.65, k-means on PCs)
#> Diversity    51 effective independent lines (of 300) · fitted Me = 68
#> Size curve   n=15: 0.34 -> 0.38 · n=30: 0.45 -> 0.48 · n=45: 0.51 -> 0.54 · n=60:
↳ 0.56 -> 0.59 · n=90: 0.63 -> 0.65 · n=120: 0.68 -> 0.69 · n=180: 0.74 -> 0.75
#>              expected accuracy (sqrt CDmean), random -> optimised
#> Next         tso_plot(diagnosis) · tso_design(data, n = 60)
```

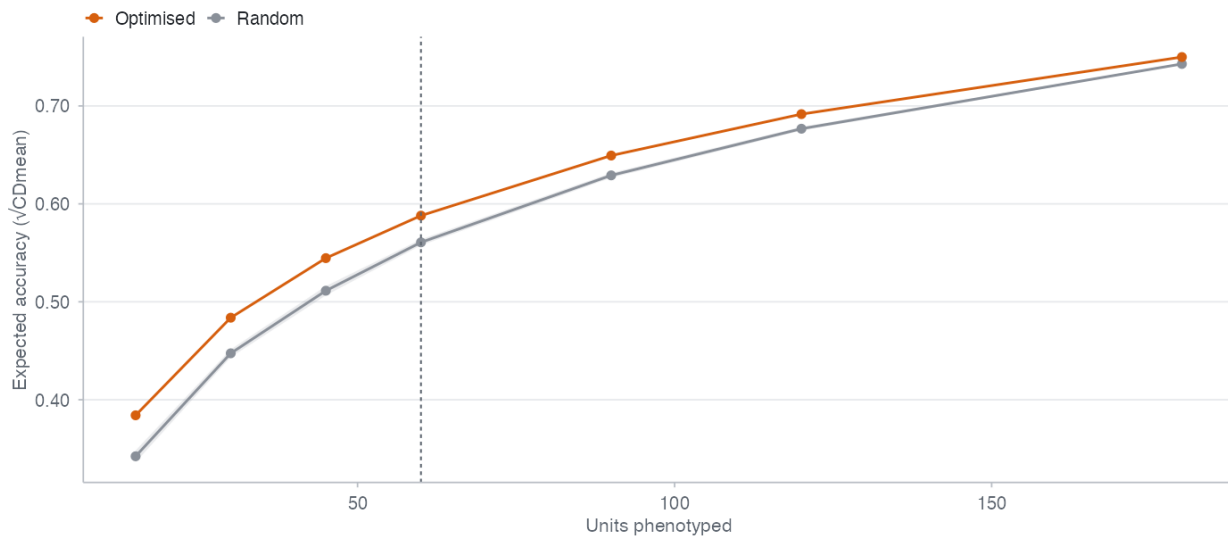
What each line means:

- **Verdict:** a traffic light for the expected gain in prediction accuracy of an optimised set over random sets of the same size. GREEN is 5% or more, AMBER 2-5%, RED under 2%. Here it is AMBER, with a gain of 4.9%.
- **Advice:** what to do about it, in words. RED means random or stratified sampling is fine, and the effort is better spent on more lines or better phenotyping.
- **Structure:** whether the lines form a continuum, mild structure or discrete groups. Here 4 discrete groups, which `tso_design()` will stratify by itself.
- **Diversity:** the effective number of independent lines (51 of 300: related lines carry overlapping information) and the fitted **Me**, the effective number of chromosome segments (68). Me is estimated from the size curve itself, not from a formula. The larger Me is, the more lines you need for a given accuracy.
- **Size curve:** expected accuracy (the square root of CDmean) for random and optimised sets at several sizes.

```
tso_plot(dg, "size_curve")
```

Accuracy vs training-set size

Fitted Me = 68 · dashed line: planned n = 60



What to look at: the gap between the two curves at your planned n (the dashed line). In this untargeted case it is small, and it shrinks further as n grows. The verdict changes when we name the lines we want to predict (below).

The diagnosis uses a single heritability ($h^2 = 0.5$ by default). Pass `target =` to diagnose a targeted design instead.

2 Step 2: how many lines do I need?

`tso_sample_size()` draws the operating curve: expected accuracy at every training-set size, relative to phenotyping every candidate. It answers “phenotype how many lines to reach 95% of the accuracy I would get by phenotyping them all?”.

```
ss <- tso_sample_size(tp)
ss
#> TSOpt sample size · 300 candidates · untargeted · averaged over h2 = 0.2, 0.5, 0.8
#> Reference      expected accuracy 0.808 (phenotyping all 300)
#> → Phenotype 223 lines to reach 95% of the accuracy of phenotyping all 300; random
  ↪ sampling would need 229.
#> Target RE      optimised random plots saved
#>              90%      167      176      9
#>              95%      223      229      6
#>              99%      282      285      3
#> Fit            logistic acc(n) = 0.775 / (1 + exp(0.41 - 0.0223 n)), RMSE 0.0323
  ↪ (smoothing only)
#> Method         exact greedy curve (optimised) · mean of 20 random sets per size (random)
```

Here reaching 95% of the full accuracy (0.808) takes 223 optimised lines against 229 random ones, a saving of only 6 plots. That is typical of the untargeted case: when the whole pool is both the training candidates and the targets, a large random set covers it well.

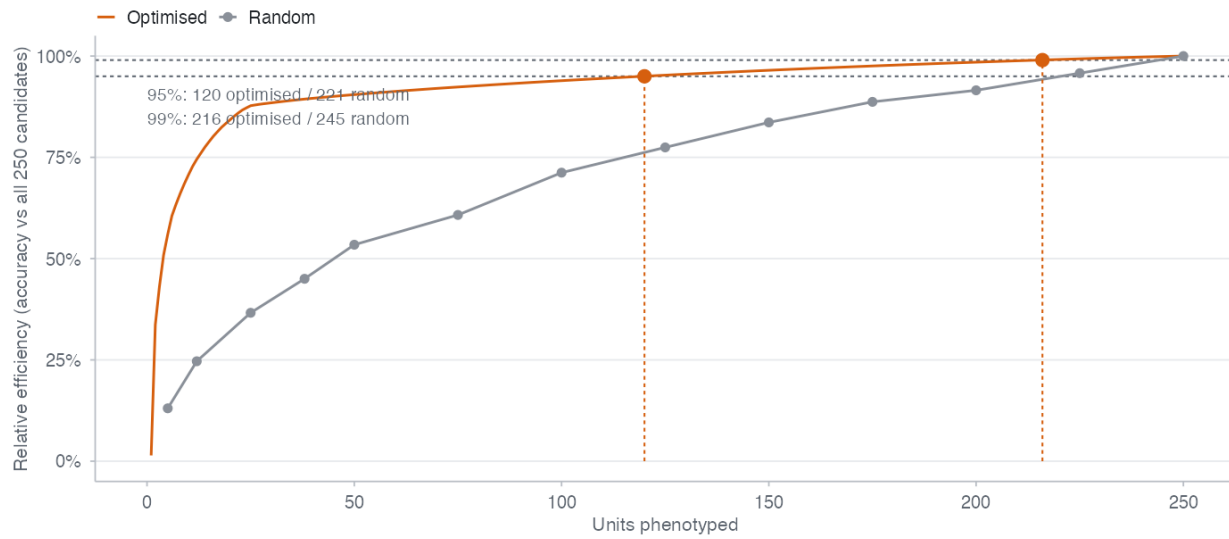
The picture changes when this season’s targets are known. Say lines L251-L300 are the new crosses we must predict; they can be genotyped but not all phenotyped.

```
targets <- ids[251:300]
ss_t <- tso_sample_size(tp, target = targets)
ss_t
#> TSOpt sample size · 250 candidates · 50 targets · averaged over h2 = 0.2, 0.5, 0.8
#> Reference      expected accuracy 0.519 (phenotyping all 250)
#> → Phenotype 120 lines to reach 95% of the accuracy of phenotyping all 250; random
  ↪ sampling would need 221.
#> Target RE      optimised random plots saved
#>              90%      45      187      142
#>              95%     120      221      101
#>              99%     216      245      29
#> Fit            logistic acc(n) = 0.495 / (1 + exp(0.68 - 0.1499 n)), RMSE 0.0200
  ↪ (smoothing only)
#> Method         exact greedy curve (optimised) · mean of 20 random sets per size (random)
```

```
tso_plot(ss_t)
```

Phenotype 120 to reach 95% of the accuracy of all 250 candidates

Exact greedy CDmean curve (averaged over h2) vs mean of 20 random sets



What to look at: with targets, 95% of the achievable accuracy takes 120 optimised lines and 221 random ones: 101 plots saved. The curve also shows where extra plots stop paying: past the knee each line adds very little.

The curve is exact for the greedy design (computed in one pass, not fitted), and by default it averages over three heritabilities, like `tso_design()`.

3 Step 3: design the training set

```
plan <- tso_design(tp, n = 60)
plan
#> TSOpt plan · inbred · 60 lines of 300 candidates · untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
#> Search greedy (robust CDmean: average over h2) (0.1 s)
#> Expected CDmean 0.346 (at h2 = 0.50) (exact, intercept projected: 0.345) vs
#> random 0.315 ± 0.004 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~4.9% accuracy over random); worth using, but do not
#> expect much
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
#> 15/15/15/15
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
#> unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
#> tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) ·
#> 90.0% of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.86% to 1.72% of the CDmean (median 1.04%) · see
#> tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

The printout, line by line:

- **Criterion:** what was optimised. CDmean is the mean reliability of the predictions of the targets (here the whole pool), with the intercept treated exactly as a fixed effect, averaged

over $h^2 = 0.2, 0.5$ and 0.8 : this is *robust CDmean*, explained below. **Search** says how the set was found: a greedy search, which takes a fraction of a second here.

- **Expected:** the CDmean of the chosen set, 0.346 (reported at $h^2 = 0.5$), against 0.315 for 100 random sets of the same size. The plan beats 100% of them.
- **Verdict:** the same traffic light as the diagnosis, now for this plan: AMBER, an expected accuracy gain of 4.9%.
- **Structure:** the groups found in the panel and how the 60 lines were allocated to them (stratification is automatic when the panel has discrete groups).
- **Coverage:** whether every target has a close relative in training. A target whose nearest training relative is below 0.05 is effectively unreachable: nothing in the training set says anything about it.
- **Divide:** candidates whose own reliability stays below 0.06: lines the design can say almost nothing about. Many of them means the panel holds outliers or a group the plan neglects.
- **Alleles:** the share of target alleles, and of 5-marker haplotypes, that also occur in the training set. An allele absent from training cannot have its effect learned (guide 4 is about this).
- **Plot value:** how much CDmean the plan loses if one plot is dropped, from the least to the most valuable plot (guide 5 shows how to use it when seed or land runs short).
- **Next:** the obvious next calls.

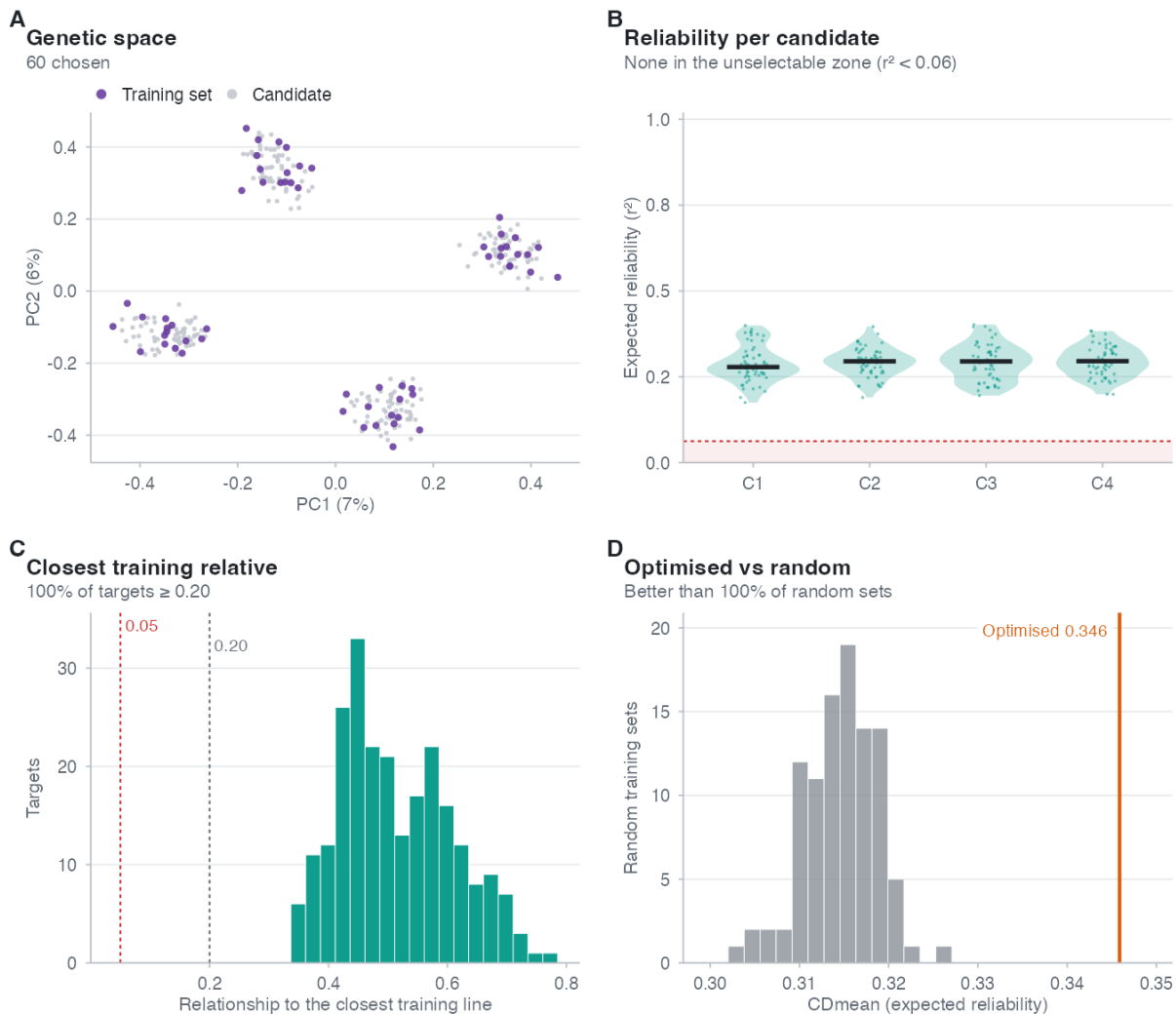
The chosen lines, in the order the search picked them, with each one's expected reliability:

```
head(plan$selected[, c("order", "id", "reliability")])
#>   order  id reliability
#> 1     1 L234  0.5752882
#> 2     2 L083  0.5864456
#> 3     3 L068  0.5986843
#> 4     4 L185  0.5846252
#> 5     5 L193  0.5694213
#> 6     6 L128  0.5555867
```

One figure summarises the plan: where the chosen lines sit in genetic space (A), how well each candidate is predicted (B, the divide), how close each target is to training (C) and the plan against random sets (D).

```
tso_plot(plan)
```

Training-set plan: 60 units • AMBER



Each panel is also available alone: `tso_plot(plan, "pca"), "divide", "coverage", "baseline"`.

4 Targeted and untargeted designs

The plan above is **untargeted**: with no target, TSOpt designs to predict the whole pool. That is the right choice for a reference panel you will use for years, or when the lines to predict are not known yet.

When you know which lines you need to predict, say so. The targets are never phenotyped, and the training set is chosen from the rest to predict them best.

```
plan_t <- tso_design(tp, n = 60, target = targets, verbose = FALSE)
plan_t
#> TSOpt plan • inbred • 60 lines of 250 candidates • 50 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
#> Search greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.229 (at h2 = 0.50) (exact, intercept projected: 0.229) vs
#> random 0.089 ± 0.021 (better than 100% of 100 random sets)
```

```
#> Verdict    GREEN optimisation pays off here: expected accuracy ~61.5% above random
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
  ↳ 12/12/25/11
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
#> Divide    0 candidates effectively unselectable (reliability < 0.06) · see
  ↳ tso_plot(plan, "divide")
#> Alleles   100.0% of target alleles present in training (0 markers not covered) ·
  ↳ 96.3% of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.15% to 8.07% of the CDmean (median 0.23%) · see
  ↳ tso_plot(plan, "value")
#> Next      tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

What to look at: the targeted plan's CDmean on the targets is 0.229, against 0.089 for random sets of the other lines: the verdict is now GREEN (62% more accuracy). The targets all sit in one group, and the allocation leans towards it.

The general rule: the more the targets differ from a random draw of the candidates, the more a targeted design pays. The diagnosis and the sample-size curve both take target =, so ask them before deciding.

5 Why robust CDmean is the default

CDmean (Rincent et al. 2012) is the expected reliability of the predictions, averaged over the targets. It depends on the heritability you assume, and before the trial nobody knows it. A set that is optimal for $h^2 = 0.5$ is not quite optimal for a trait that turns out to have $h^2 = 0.2$. **Robust CDmean** averages the criterion over $h^2 = 0.2, 0.5$ and 0.8 , and so chooses a set that is good across the plausible range:

```
round(plan$value$cd_by_h2, 3)
#> h2=0.2 h2=0.5 h2=0.8
#> 0.178 0.346 0.482
```

This was chosen as the default after a head-to-head benchmark on six public panels (maize, two rice panels, sorghum, switchgrass and spruce; 33 scenarios, real phenotypes, GBLUP predictions). Predictive ability (PA) is the correlation between predictions and phenotypes on the lines not in training:

Method	PA gain over random	Beats random	Median time
TSOpt robust CDmean (default)	+0.069	91%	0.03 s
TSOpt CDmean at $h^2 = 0.5$	+0.064	85%	0.02 s
GA + SA from random starts (TrainSel-style)	+0.061	85%	6.8 s
TSDFGS r-score	+0.044	82%	69 s
STPGA genetic algorithm (CDmean)	+0.044	76%	68 s
Stratified random	+0.006	61%	0 s

Two lessons from it. A higher criterion value is not a better prediction: harder searches reach a higher CDmean at the assumed h^2 , but predict no better, because they over-fit a heritability that is wrong for most traits. And hedging over h^2 wins. You can still give one value ($h^2 = 0.3$) when you know it, or your own grid ($h^2 = c(0.1, 0.3, 0.5)$).

6 Non-additive models: total or breeding value?

With a non-additive genetic model from `tso_data(kernels = )` (guide 1), a design can predict two different things:

- **The total genetic value** (`value = "total"`, the default): what a line, clone or variety is worth itself. Every kernel contributes.
- **The breeding value** (`value = "breeding"`): the additive part, what a parent passes on. Use it when the lines are chosen as parents. The data still carry every kernel (epistasis is part of what the plots measure), but only the additive part is predicted.

```
tp_aa <- tso_data(ex("example_panel.vcf.gz"), kernels = c("A", "AA"), verbose = FALSE)
plan_tv <- tso_design(tp_aa, n = 60, target = ids[251:300], verbose = FALSE)
plan_bv <- tso_design(tp_aa, n = 60, target = ids[251:300], value = "breeding", verbose = FALSE)
c(same_lines = length(intersect(plan_tv$selected$id, plan_bv$selected$id)),
  CD_breeding_of_bv_plan = round(plan_bv$value$cdmean, 3),
  CD_breeding_of_tv_plan = round(tso_score(tp_aa, plan_tv$selected$id, target =
    ↪ ids[251:300],
                                value = "breeding")["cdmean"], 3))
#>          same_lines CD_breeding_of_bv_plan CD_breeding_of_tv_plan
#>          56.000      0.165      0.165
```

With the robust shares, the design averages over three share scenarios times the three heritabilities; the plan's Scenarios line reports each one.

How much does it matter? We simulated traits with 30% or 50% of the genetic variance additive x additive, on structured and continuous simulated panels and on the example panel (720 paired runs, benchmark/kernels_benchmark.R), and predicted the targets by GBLUP with the true model:

Design	Accuracy, total value	Accuracy, breeding value
Random	0.371	0.400
Additive default	0.427	0.448
Kernels A + AA (robust shares)	0.430	0.446
Kernels, value = "breeding"	0.430	0.451

The additive default is already robust to epistasis: modelling it in the design adds about +0.003 in accuracy (not significant), against +0.055 for optimising at all. Aiming at breeding values gives a small but consistent gain for breeding values (+0.005). So keep the default for an ordinary design. Use kernels when you want the plan's reliability reported under the model you believe, when you choose parents (`value = "breeding"`), or when candidates are not genotyped yet (the single-step H, guide 1).

7 Other criteria

You rarely need to change the criterion. `tso_criteria()` lists all 18 and says when each one is worth using:

```
crit <- tso_criteria()
knitr::kable(crit[crit$tier != "Classical", c("criterion", "tier", "use_when")],
             row.names = FALSE)
```

criterion	tier	use_when
cdmean	Recommended	almost always (the default)
pevmean	Specialist	very large pools, ranking identical to CDmean
cdpop	Specialist	targets are crosses or families, not lines
reliability	Specialist	breeding values themselves, not deviations, are what you report
cdmin	Specialist	no target may be left unpredictable; genebanks, pre-breeding
pevmax	Specialist	as cdmin, on the variance scale
gain	Specialist	strong truncation selection (top 1-10%)
coverage	Model-free	model-free coverage; very large or poorly modelled panels
haplotype	Model-free	genebank cores and reference panels; rare haplotypes must be learnable
maximin	Model-free	model-free diversity
pam	Model-free	model-free representativeness
stratified	Model-free	baseline for structured panels
random	Baseline	baseline
kinship	Baseline (harmful)	never, except to show why not

The four classical optimal-design criteria (dopt, aopt, eopt, rscore) are left out of the table above; `tso_criteria("Classical")` shows them. Goals in breeder language, such as `goal = "gain"` or `goal = "reference_panel"`, are covered in **TSOpt guide 3: Designing for your programme**.

8 What exactly did it do? `tso_explain()`

`tso_explain()` prints every choice behind a plan in plain words: the criterion and its formula, the heritability, targets, candidates, constraints, stratification and search. It also returns the full specification, so the plan can be reproduced.

```
tso_explain(plan, steps = FALSE)
#> How this plan was made
#> System      inbred
#> Goal        accuracy -> CDmean: the mean coefficient of determination of target
#> contrasts (each target minus the target mean),  $CD_i = 1 - c_i' A c_i / c_i' K c_i$ 
#> Heritability robust: the criterion is averaged over  $h2 = 0.2, 0.5, 0.8$  (nobody
#> knows  $h2$  before the trial)
#> Targets      untargeted (whole pool) (300 units)
#> Candidates    300 units
#> Size          n = 60
#> Constraints   group_quota = 15/15/15/15
#> Structure     discrete groups -> stratified
#> Search        greedy (robust CDmean: average over  $h2$ )
#> Fixed effects intercept handled exactly (flat prior)
#> Reproduce     seed = 1, TSOpt 0.5.0
```

With `steps = TRUE` (the default) it adds the algebra step by step. The vignette *CDmean by hand*, then with TSOpt (`vignette("by-hand", package = "TSOpt")`) does the same calculation in plain base R.

9 Hand it over: `tso_export()` and `tso_report()`

`tso_export()` writes the list for the field team as a CSV and returns it as a table:

```
field <- tso_export(plan, file.path(tempdir(), "trial_2027.csv"))
head(field, 3)
#>   order  id line cost reliability forced
#> 1     1  L234 L234    1   0.5752882 FALSE
#> 2     2  L083 L083    1   0.5864456 FALSE
#> 3     3  L068 L068    1   0.5986843 FALSE
```

Field	Meaning
order	the order in which the search picked the line; if you must cut the list, cut from the bottom
id, line	the unit and the line it belongs to (they differ for hybrids and multi-environment units)
group, family, env, parent1, parent2	shown when the data have them
cost	the unit's cost (1 when no costs were given)
reliability	the expected reliability of that line's own genetic value after the trial
forced	TRUE for lines forced in with <code>must_include</code> (checks)

`tso_export(plan, what = "all")` lists every unit instead, with its role (training, candidate or target) and expected reliability: useful to see how well each target will be predicted.

`tso_report()` writes a self-contained one-page HTML report for a programme manager: the decision, the expected gain over random, the risks and the figures. It needs no pandoc or LaTeX.

```
tso_report(plan, "trial_2027.html", diagnosis = dg,
           title = "Training set 2027", author = "Your name")
```

10 The whole workflow in 8 lines

Here `targets` is the vector of line IDs you must predict; leave `target = out` for an untargeted design.

```
library(TSOpt)
tp <- tso_data("my_panel.vcf.gz")           # read, QC, kinship
dg <- tso_diagnose(tp, n = 60)              # worth optimising?
ss <- tso_sample_size(tp, target = targets) # how many lines?
plan <- tso_design(tp, n = 60, target = targets)
tso_plot(plan)                             # check it
tso_export(plan, "trial_2027.csv")          # field list
tso_report(plan, "trial_2027.html", diagnosis = dg)
```

11 Next

- **TSOpt guide 3: Designing for your programme:** budgets and costs, checks, lost seed and last season's data, quotas, hybrids, multiple environments and traits, other goals, the field layout, whom to genotype and training across programmes.

- **TSOpt guide 4: Major genes, haplotypes and fixed effects:** when a major gene is present, and how to make sure the training set carries its alleles.
- **TSOpt guide 5: Check, decide and plan ahead:** comparing plans, how sure the advantage is, the value of each plot and the economic size, scoring plans once the phenotypes are in, the next stage and the next season.